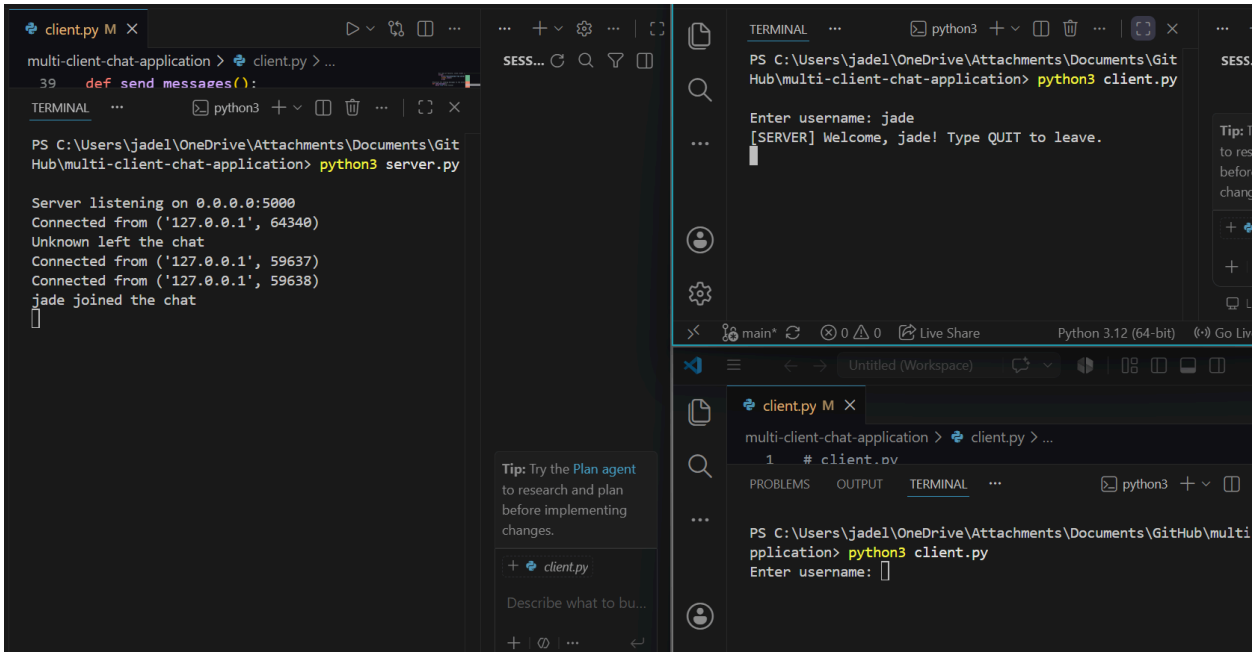
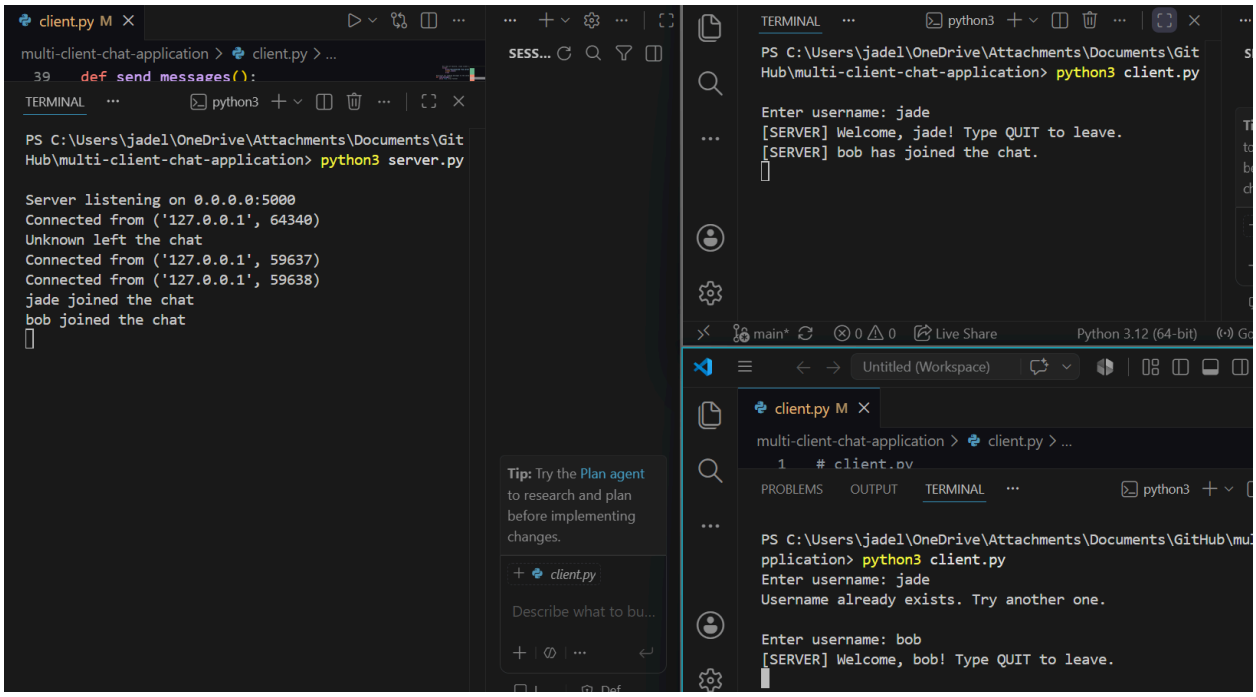
Problem (Multi-Client Chat Application Using TCP Sockets in Python).
In this assignment,
you will develop a multi-client chat application using TCP sockets in Python. The system consists of a centralized server and multiple clients communicating through the client-server model. Each client connects directly to the server, sends chat messages to the server, and the server then
forwards (broadcasts) those messages to all other connected clients. Clients do not communicate with each other directly; all message traffic flows through the server. The server must support multiple simultaneous client connections and handle message delivery in real time. A client can join or leave the chat without affecting the remaining clients' communication. This assignment helps you to practise TCP socket programming, concurrent network communication using multi threads, and basic application-layer protocol design in Python.
Requirements: server should support multiple clients concurrently using python threading module and each client communicates with the server in an individual thread. When client connects to the server, the server will ask for a unique username (to distinguish from other clients). The server stores the username. If it exists, it should ask for another username. When a client joins or leaves the system, server should notify others. A client leaves with quit QUIT message and then server should remove him and close his socket. The client should always wait for or read the keyboard input. Students should only use Python3 socket and threading modules. You may not use external networking libraries.
Submission: (1) client.py and server.py; (2) README file explaining how to run the program and the design overview; (3) screenshots show multiple connected clients, messages for two clients and one server, and how the disconnection is handled.

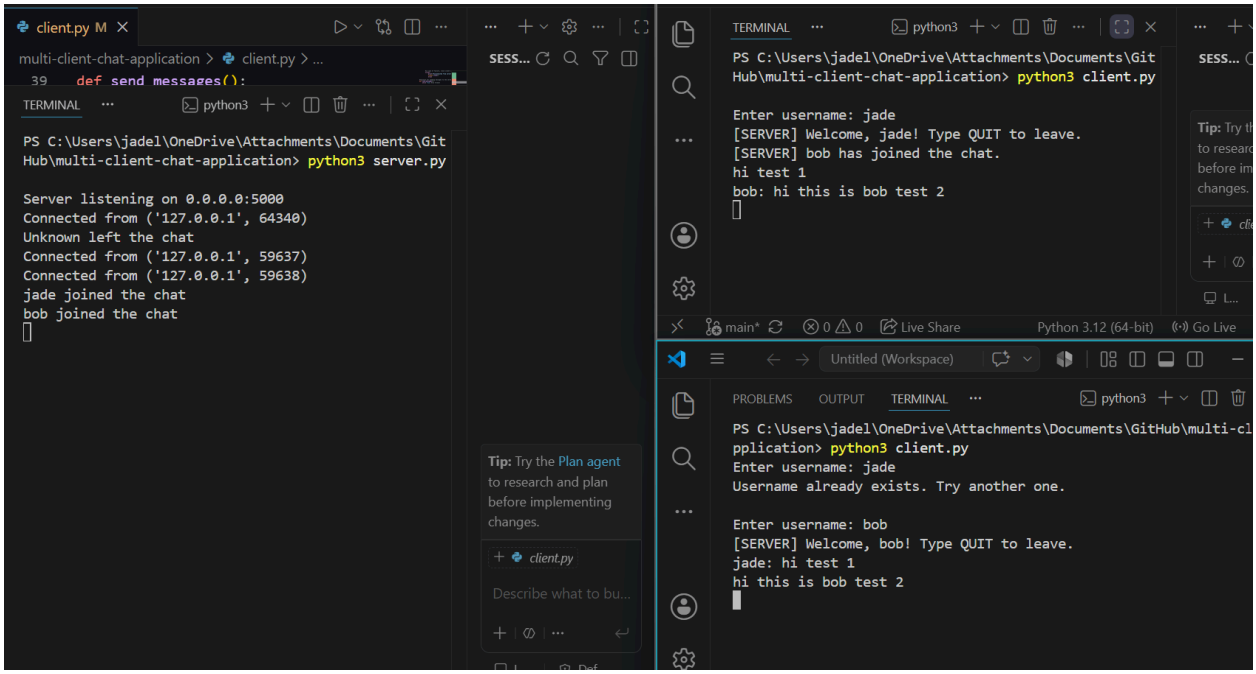
One person joining chat



2nd person joining chat + username already existing



Test messages



Leaving chat